# Dokumentation SMP Projekt

Peter Wienken, Aaron Weis

6. März 2026

## Inhaltsverzeichnis

**1 Audio Import** **2**

**2 FFT Analyse** **2**
2.1 FFT Algorithm . . . . . 2
2.2 Windowing . . . . . 3

**3 FLUX Detection** **3**
3.1 Calculation of Logarithmic Magnitude Spectrum . . . . . 3
3.2 Calculation of Spectral Flux . . . . . 3
3.3 Half-Wave Rectification . . . . . 4

**4 Onset Detection** **5**
4.1 Adaptive Threshold and offset . . . . . 6
4.2 Identification of Peaks (Peak Picking) . . . . . 8
4.3 Practical Application . . . . . 8

**5 MIDI Export** **9**

**6 Deep Learning based Source Separation** **9**

**7 Fazit** **9**

# 1 Audio Import

# 2 FFT Analyse

## 2.1 FFT Algorithm

The Fast Fourier Transform (FFT) algorithm is used for computing the Discrete Fourier Transform (DFT) of a sequence. The DFT of a sequence $x(n)$ of length $N$ is defined as:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N} \tag{1}$$

For every frequency bin $k$, the DFT computes a complex number $X(k)$ that represents the amplitude and phase of the corresponding frequency component in the input signal. To calculate $N$ coefficients, $O(N^2)$ operations are required.This is because for each of the N output values, a sum of N terms must be computed.

The FFT algorithm separates the DFT into smaller DFTs, by recursively dividing the input sequence into even and odd indexed elements.

$$X(k) = \sum_{n=0}^{N/2-1} x(2n)e^{-j2\pi k(2n)/N} + \sum_{n=0}^{N/2-1} x(2n+1)e^{-j2\pi k(2n+1)/N} \tag{2}$$

For better understanding, the even and odd indexed elements can be represented as, by using the substitution:

$$X_{even}(k) = \sum_{n=0}^{N/2-1} x(2n)e^{-j2\pi kn/(N/2)} \tag{3}$$

and

$$X_{odd}(k) = \sum_{n=0}^{N/2-1} x(2n+1)e^{-j2\pi kn/(N/2)} \tag{4}$$

$$X(k) = X_{even}(k) + e^{-j2\pi k/N} X_{odd}(k) \tag{5}$$

Because of the peridicity of the complex exponential function, to different values of $k$ can be calculated by:

$$X(k + N/2) = X_{even}(k) - e^{-j2\pi k/N} X_{odd}(k) \tag{6}$$

And

$$X(k) = X_{even}(k) + e^{-j2\pi k/N} X_{odd}(k) \tag{7}$$

By splitting the DFT into tow smaller DFTs, the number of operations is reduced to $O(N \log N)$, which makes the FFT algorithm much more efficient for large input sizes.

### 2.2 Windowing

## 3 FLUX Detection

### 3.1 Calculation of Logarithmic Magnitude Spectrum

The logarithmic magnitude spectrum is calculated using the formula:

$$L(k) = \log_{10}(|X(k)|) \tag{8}$$

It transforms the linear magnitude spectrum into a logarithmic scale, which is more aligned with human auditory perception.

### 3.2 Calculation of Spectral Flux

The spectral flux is calculated as the difference between the logarithmic magnitude spectra of consecutive frames:

$$F(n) = \sum_{k=0}^{N-1} L_n(k) - L_{n-1}(k) \tag{9}$$

For each frame $n$, the spectral flux $F(n)$ is computed by summing the squared differences of the logarithmic magnitude spectra $L_n(k)$ and $L_{n-1}(k)$ across all frequency bins $k$. The frames are typically stored in a 2D array where each column represents a frame and each row represents a frequency bin. The difference between consecutive frames can be represented as:

$$\Delta L(k) = L_n(k) - L_{n-1}(k) \tag{10}$$

Example of a 2D array representing frames and frequency bins:

$$\mathbf{L} = \begin{bmatrix} 1 & 4 & 1 & 6 \\ 3 & 2 & 2 & 1 \\ 1 & 5 & 3 & 6 \end{bmatrix}$$

The resulting difference between consecutive frames is:

$$\Delta\mathbf{L} = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 2 & 1 \\ 5 & 3 & 6 \end{bmatrix} - \begin{bmatrix} 1 & 4 & 1 \\ 3 & 2 & 2 \\ 1 & 5 & 3 \end{bmatrix} = \begin{bmatrix} 3 & -3 & 5 \\ -1 & 0 & -1 \\ 4 & -2 & 3 \end{bmatrix}$$

Then, the spectral flux is computed by summing these differences across all frequency bins for each frame:

$$F(n) = \sum_k |\Delta L(k)| \tag{11}$$

$$\mathbf{F} = \begin{bmatrix} |3| + |(-1)| + |4| & |(-3)| + |0| + |(-2)| & |5| + |(-1)| + |3| \end{bmatrix}^T = \begin{bmatrix} 8 & 5 & 9 \end{bmatrix}^T$$

The Code to calculate the spectral flux from the differences is as follows:

```
flux = np.sum((calculated_FLUX[:, 1:] - calculated_FLUX
    [:, :-1]))
```

## 3.3 Half-Wave Rectification

Half-wave rectification is applied to the spectral flux to retain only positive values, which correspond to increases in spectral energy. This is done by setting all negative values to zero:

$$F_{rect}(n) = \max(0, F(n)) \tag{12}$$

This step ensures that only significant increases in spectral energy are considered for further analysis.

For example, given the spectral flux values:

$$\Delta L = \begin{bmatrix} -2 & 2 & -4 \\ 3 & 1 & 5 \\ -1 & 4 & -6 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 0 \\ 3 & 1 & 5 \\ 0 & 4 & 0 \end{bmatrix}$$

And Summed up:

$$F = \begin{bmatrix} 0+3+0 & 2+1+4 & 0+5+0 \end{bmatrix}^T = \begin{bmatrix} 3 & 7 & 5 \end{bmatrix}^T$$

# 4 Onset Detection

Onset detection is an essential processing step within the pipeline. The goal of this procedure is to identify the exact timings (onsets) of musical events. In the case of drum detection, drum hits are identified, from a novelty function, which is the calculated positive spectral flux.
Consequently, a peak is only registered if three criteria are met simultaneously:

1. **Amplitude:** The signal must be loud enough and exceed the local adaptive threshold.

2. **Local Maximum:** The considered point must have a larger value than its direct temporal neighbors (the frame before and after).

3. **De-bouncing (Minimum Distance):** A single musical event can cause fluctuations over several frames. To avoid multiple detections of the same beat, a defined waiting time (`wait`) must be maintained between two detected peaks.

## 4.1 Adaptive Threshold and offset

An adaptive threshold follows the rough envelope of the signal. Typically, a moving average is calculated over a time window of a defined size for this purpose. The moving average serves to smooth the spectral flux and forms the basis of the adaptive threshold. Mathematically, the filter can be described as a discrete convolution of the input signal with a filter kernel.
Let $x[n]$ be the discrete-time input signal (the spectral flux at frame $n$) and $N$ the window size (`window_size`). The output of the filter $y[n]$ is the unweighted mean of the $N$ adjacent samples.
The centered calculation (corresponding to the `mode='same'` parameter in the code) can be expressed by the following sum formula:

$$y[n] = \frac{1}{N} \sum_{k=-\lfloor \frac{N}{2} \rfloor}^{\lfloor \frac{N-1}{2} \rfloor} x[n+k] \tag{13}$$

In signal processing, this operation is typically formulated as a discrete convolution. The signal $x[n]$ is convolved with the impulse response (the kernel) $h[n]$:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k] \tag{14}$$

The filter kernel $h[n]$ is a rectangular window of length $N$, where all values are equally weighted. This corresponds to the code line `kernel = np.ones(window_size) / window_size`:

$$h[n] = \begin{cases} \frac{1}{N} & \text{for } 0 \leq n < N \\ 0 & \text{otherwise} \end{cases} \tag{15}$$

Through this convolution operation, short-term noise is suppressed and the fundamental dynamic envelope (the trend) of the signal is extracted, onto which the offset for the final threshold is subsequently added. To prevent background noise (noise floor) from being falsely detected as a peak, an offset (delta) is added to this average. This offset can be static or dynamically adapt to signal blocks.
In the present implementation, the moving average is realized by a mathematical convolution (`np.convolve`) of the flux with a uniformly distributed kernel. The offset is calculated and added block-wise:

```python
def calculate_adaptive_threshold(self, window_size):
    # Filter kernel for moving average
    kernel = np.ones(window_size) / window_size
```

```python
    moving_avg = np.convolve(self.flux, kernel, mode='same')

    # Calculate and add offset (delta)
    deltas = self.calculate_delta(window_size, 3)
    for i in range(len(moving_avg)):
        delta_index = int(i / window_size)
        if delta_index < len(deltas):
            moving_avg[i] += deltas[delta_index]

    self.threshold_curve = moving_avg
    return self.threshold_curve

def calculate_delta(self, window_size, multiplier):
    # Block-wise calculation of the noise offset
    len_flux = len(self.flux)
    delta_values = np.zeros(int(np.ceil(len_flux/window_size)))
    for i in range(len(delta_values)):
        start = i * window_size
        end = min((i + 1) * window_size, len_flux)
        delta_values[i] = np.mean(self.flux[start:end]) * multiplier
    return delta_values
```

## 4.2 Identification of Peaks (Peak Picking)

Once the threshold curve is available, the algorithm iterates through the signal. At each point, it checks whether the current value exceeds the threshold, whether it is the tip of a local hill, and whether the time distance to the last detected peak is large enough (de-bouncing).
This can be implemented very directly as a loop with conditional checks.

```
def find_peaks(self, window_size, wait):
    thresholds = self.calculate_adaptive_threshold(
        window_size)
    found_peaks = []
    last_peak_index = -wait  # Initialization for
        debouncing
    n_frames = len(self.flux)

    for i in range(1, n_frames - 1):
        current_val = self.flux[i]
        current_thresh = thresholds[i]

        # Condition 1: Above the threshold?
        if current_val > current_thresh:
            # Condition 2: Local maximum?
            if current_val > self.flux[i-1] and
                current_val > self.flux[i+1]:
                # Condition 3: Debouncing time expired?
                if (i - last_peak_index) > wait:
                    found_peaks.append(i)
                    last_peak_index = i

    self.onsets = np.array(found_peaks)
    return self.onsets
```

## 4.3 Practical Application

The parameters for window size (`window_size`) and waiting time (`wait`) depend heavily on the characteristics of the signal to be analyzed. For percussive, low-frequency instruments like a kick drum, different parameters are often chosen compared to high-frequency, fast instruments like a hi-hat.
In an exemplary use case, a window size of 16 is used for the kick and snare, for instance, while the more densely played hi-hat is smoothed with a window size of 32, both maintaining the same de-bouncing wait time of 4 frames.

# 5 MIDI Export

# 6 Deep Learning based Source Separation

# 7 Fazit

Hier steht das Fazit.